

# 核心算法代码逐行精讲

西安地铁换乘最优路径计算器 —— script.js 核心函数逐行讲解，答辩时可以直接照着说。 ● 蓝色标注 = JS 代码解释 ● 橙色标注 = C 语言对照

## 一、图数据模型：graph.json 的结构

在讲算法之前，必须先搞懂数据是怎么组织的。graph.json 是整个系统的数据基础。

graph.json 包含两个核心数组：nodes（节点）和 edges（边）。

### 1.1 nodes 数组 —— 图的节点

每个节点代表"某个站名的某条线路"，不是物理站点。换乘站会拆成多个节点。

```
{
  "id": 131,           // 节点唯一编号，0~288
  "station": "小寨",   // 站名（同一站名可出现多次）
  "line": "3号线",     // 该节点所属线路
  "lat": 34.2271,      // 纬度（WGS84原始坐标）
  "lon": 108.9487      // 经度
}
```

● 小寨站同时属于2号线和3号线，所以有两个节点：id=95(2号线) 和 id=131(3号线) ● 它们之间通过一条换乘边连接，权重5分钟 ● 这样设计是为了让 Dijkstra 能追踪"当前在哪条线上"，从而正确计算换乘

### 1.2 edges 数组 —— 图的边

```
{
  "from": 131,         // 起点节点id
  "to": 132,           // 终点节点id
  "weight": 2.5,        // 边的权重（分钟）
  "line": "3号线",     // 所属线路
  "is_transfer": 0      // 0=行驶边，1=换乘边
}
```

● 行驶边：相邻两站之间的乘车时间，通常1.5~3分钟 ● 换乘边：同一站名不同线路节点之间的连接，权重5分钟（含2分钟等待） ● `is_transfer` 是关键标志，Dijkstra 用它来计算换乘次数

### ● C语言对照：如果用C写，`nodes`和`edges`对应的结构体如下

```
// C 语言中，nodes 数组的每个元素是这样的结构体
typedef struct {
    int id;           // 节点唯一编号
    char station[32]; // 站名
    char line[16];    // 所属线路
    double lat;       // 纬度
    double lon;       // 经度
} Node;

// edges 数组的每个元素
typedef struct {
    int from;         // 起点节点id
    int to;           // 终点节点id
    double weight;    // 边的权重（分钟）
    char line[16];    // 所属线路
    int is_transfer;  // 0=行驶边，1=换乘边
} Edge;

// JS 的 JSON 对象 ≈ C 的结构体实例
// JS: { id: 131, station: "小寨", line: "3号线" }
// C: Node n = { 131, "小寨", "3号线", 34.2271, 108.9487 };
// 区别: C 需要提前定义 struct 类型, JS 随用随写
```

### 答辩话术：

"我们的图有289个节点、636条边。为什么不是250个节点？  
因为换乘站按'站名+线路'拆分，比如小寨站拆成2号线节点和3号线节点，中间用换乘边连接。这样Dijkstra扩展时能知道当前在哪条线上，才能正确计算换乘次数和换乘等待时间。"

## 二、buildAdjList —— 构建邻接表

邻接表是 Dijkstra 运行时查询"某个节点能到达哪些邻居"的数据结构。

## 2.1 代码逐行讲解

```
function buildAdjList() {
```

- 定义函数，无参数，因为 graphData 是全局变量

```
if (typeof ScheduleConfig !== "undefined") {
```

- 如果时段调度模块存在，用它的方法构建（会自动调整权重）

```
adjList = ScheduleConfig.buildAdjustedAdjList(graphData);
```

- 调用调度模块的方法，根据当前时段（高峰/平峰）调整边权

```
} else {
```

- 如果没有调度模块，手动构建基础邻接表

```
adjList = {};
```

- 初始化空对象，key是节点id，value是该节点的邻居数组

```
for (var i = 0; i < graphData.nodes.length; i++) {
```

- 遍历所有节点

```
adjList[graphData.nodes[i].id] = [];
```

- 为每个节点初始化空数组，用来存邻居信息

```
}
```

- 循环结束，所有节点都有了空的邻居列表

```
for (var j = 0; j < graphData.edges.length; j++) {
```

- 遍历所有边

```
var e = graphData.edges[j];
```

- 取出第j条边

```
adjList[e.from].push({
```

- 把这条边的信息加到起点节点的邻居列表中

```
to: e.to,
```

- 邻居节点id

```
weight: e.weight,
```

- 边的权重（分钟）

```
line: e.line,
```

- 所属线路

```
is_transfer: e.is_transfer
```

## ● 是否换乘边 (0或1)

```

    });
  }
}
}

```

## 2.2 邻接表长什么样?

```

adjList[131] = [
    // 小寨(3号线)的邻居
    { to: 132, weight: 2.5, line: "3号线", is_transfer: 0 }, // → 大雁塔(3号线)
    { to: 130, weight: 2.0, line: "3号线", is_transfer: 0 }, // → 吉祥村(3号线)
    { to: 95, weight: 5.0, line: null, is_transfer: 1 } // → 小寨(2号线) 换乘
]

```

● 行驶边的 line 有值, 换乘边的 line 为 null ● 查询邻居时, adjList[nodeId] 就能拿到所有可达节点

## ● C语言对照: 邻接表在C中的实现

```

// C 中邻接表用指针数组 + 链表实现
typedef struct AdjNode {
    int to;
    double weight;
    char line[16];
    int is_transfer;
    struct AdjNode* next;
} AdjNode;

AdjNode* adjList[289]; // C: 指针数组, 每个元素是链表头
// JS: adjList = {}    // JS: 对象, 每个key对应一个数组

// C 中添加边
AdjNode* newNode = (AdjNode*)malloc(sizeof(AdjNode));
newNode->to = e.to;
newNode->weight = e.weight;
newNode->next = adjList[e.from]; // 头插法
adjList[e.from] = newNode;

```

```
// JS 中添加边（本项目实际代码）
adjList[e.from].push({ to: e.to, weight: e.weight, line: e.line, is_transfer: e.is_transfer });
// push ≈ C 的链表头插法，但 JS 不需要手动分配内存和维护指针
```

## 三、MinHeap —— 最小堆（优先队列）

最小堆是 Dijkstra 高效运行的关键数据结构，用来快速取出“当前距离最小”的节点。

### 3.1 数据结构：一维数组存储完全二叉树

堆的存储：this.heap = []

下标关系（从0开始）：

节点 i 的父节点 =  $\text{floor}((i-1)/2)$

节点 i 的左子节点 =  $2*i + 1$

节点 i 的右子节点 =  $2*i + 2$

举例：堆中有5个元素 [10, 20, 30, 40, 50]

下标0(10) 的左子=1(20)，右子=2(30)

下标1(20) 的左子=3(40)，右子=4(50)

下标3(40) 的父= $\text{floor}((3-1)/2)=1(20)$

堆性质：每个父节点的cost ≤ 子节点的cost

→ heap[0] 永远是最小值

### ● C语言对照：MinHeap 的 C 实现方式

```
// C 中用结构体 + 函数指针（或独立函数+传指针）
typedef struct {
    int nodeId;
    double cost;
    double totalTime;
    int transfers;
} HeapNode;

typedef struct {
    HeapNode heap[289];
    int size;
} MinHeap;
```

```
// JS: function MinHeap() { this.heap = []; }
// C: 用 typedef struct 定义, JS 用构造函数
// JS: this.heap → C: h->heap (通过指针访问)
// JS: this.heap.push(node) → C: h->heap[h->size++] = node;
// JS: this.heap.pop() → C: h->heap[--h->size];
// JS: this.heap.length → C: h->size
```

### 3.2 insert —— 插入操作 (上浮)

```
MinHeap.prototype.insert = function (node) {
```

● node = {nodeId: 节点id, cost: 优先级值, totalTime: 总时间, transfers: 换乘数} ● C对照: void MinHeap\_insert(MinHeap\* h, HeapNode node) —— C 要传指针 h, JS 用 this 自动绑定

```
this.heap.push(node);
```

● 先把新元素放到数组末尾 ● C对照: h->heap[h->size++] = node; —— C 手动维护 size, JS 的 push 自动处理

```
var i = this.heap.length - 1;
```

● i 是新元素的下标 (最后一个位置) ● C对照: int i = h->size - 1; —— 完全一致

```
while (i > 0) {
```

● 只要还没到根节点, 就继续上浮

```
var parent = Math.floor((i - 1) / 2);
```

● 计算父节点下标 ● C对照: int parent = (i - 1) / 2; —— C 整数除法自动取整, JS 需要 Math.floor()

```
if (this.heap[parent].cost <= this.heap[i].cost) break;
```

● 如果父节点已经  $\leq$  当前节点，堆性质满足，停止 ● C对照： `if (h->heap[parent].cost <= h->heap[i].cost) break;` —— `this.heap`  $\rightarrow$  `h->heap`

```
var tmp = this.heap[parent];
this.heap[parent] = this.heap[i];
this.heap[i] = tmp;
```

● 否则交换父节点和当前节点 ● C对照： `HeapNode tmp = h->heap[parent]; h->heap[parent] = h->heap[i]; h->heap[i] = tmp;` ● 区别：C 要声明 `HeapNode tmp`（带类型），JS 只写 `var tmp`（无类型）

```
i = parent;
```

● 继续从父位置往上检查

```
}
};
```

● insert结束。时间复杂度 $O(\log n)$ ，因为最多上浮树的高度层

### 答辩话术：

"insert就是把新元素放到数组末尾，然后和父节点比较，  
如果比父节点小就交换，一直往上'浮'，直到父节点比它小或者到了根。  
因为完全二叉树的高度是 $\log n$ ，所以最多比较 $\log n$ 次，时间复杂度 $O(\log n)$ 。"

## 3.3 extractMin —— 取出最小值（下沉）

```
MinHeap.prototype.extractMin = function () {
```

● 取出堆顶（最小值）

```
if (this.heap.length === 0) return null;
```

- 空堆返回null

```
var min = this.heap[0];
```

- 保存堆顶元素，这是要返回的最小值

```
var last = this.heap.pop();
```

- 弹出数组最后一个元素

```
if (this.heap.length > 0) {
```

- 如果弹出后堆不空，需要调整

```
this.heap[0] = last;
```

- 把最后一个元素放到堆顶位置

```
var i = 0;
```

- 从堆顶开始下沉

```
while (true) {
```

- 下沉循环

```
var left = 2 * i + 1;
```

### ● 左子节点下标

```
var right = 2 * i + 2;
```

### ● 右子节点下标

```
var smallest = i;
```

### ● 假设当前节点是最小的

```
if (left < this.heap.length && this.heap[left].cost < this.heap[smallest].cost) {
    smallest = left;
}
```

### ● 如果左子节点存在且比当前小，记录左子为最小

```
if (right < this.heap.length && this.heap[right].cost < this.heap[smallest].cost) {
    smallest = right;
}
```

### ● 如果右子节点存在且比当前最小还小，记录右子为最小

```
if (smallest === i) break;
```

### ● 如果当前节点已经是最小的，堆性质恢复，停止

```
var tmp = this.heap[smallest];
this.heap[smallest] = this.heap[i];
this.heap[i] = tmp;
i = smallest;
```

- 否则交换当前节点和最小的子节点，继续从子节点位置往下检查

```

    }
  }
  return min;
};

```

- extractMin结束。时间复杂度 $O(\log n)$ ，最多下沉树的高度层

### 答辩话术：

"extractMin先保存堆顶（最小值），然后把数组最后一个元素移到堆顶，再和两个子节点比较，如果比子节点大就和较小的子节点交换，一直往下'沉'。同样最多走 $\log n$ 层，时间复杂度 $O(\log n)$ 。"

## 3.4 isEmpty —— 判空

```

MinHeap.prototype.isEmpty = function () {
  return this.heap.length === 0;
};

```

- 判断堆是否为空，Dijkstra主循环用 while (!heap.isEmpty()) 控制退出
- C对照：int MinHeap\_isEmpty(MinHeap\* h) { return h->size == 0; } —— JS用 .length，C用 .size

## 四、dijkstra —— 核心最短路径算法

这是整个项目最核心的函数，逐行讲清楚。

### 4.1 函数签名和参数

```

function dijkstra(startName, endName, mode) {

```

- startName: 起点站名，如"小寨"
- endName: 终点站名，如"华清池"
- mode: 0=时间最短, 1=换乘最少, 2=综合最优
- C对照：void dijkstra(char\* startName, char\* endName, int mode) —— C 要声明参

## 数类型

## 4.2 找到起点和终点对应的所有节点

```
var nodes = graphData.nodes;
```

- 取出所有节点

```
var n = nodes.length;
```

- 节点总数, n=289

```
var startNodes = [];  
var endNodes = [];
```

- 存储起点/终点站名对应的所有节点（换乘站会有多个）

```
for (var i = 0; i < n; i++) {  
    if (nodes[i].station === startName) startNodes.push(i);  
    if (nodes[i].station === endName) endNodes.push(i);  
}
```

- 遍历所有节点，站名匹配的加入对应数组（注意：push的是节点在数组中的下标i，不是节点id）

```
if (startNodes.length === 0 || endNodes.length === 0) {  
    return { path: [], error: "No path found" };  
}
```

- 找不到起点或终点节点，返回空路径和错误信息

## 4.3 初始化所有数组

```
var endSet = {};
for (var e = 0; e < endNodes.length; e++) {
    endSet[endNodes[e]] = true;
}
```

● 用对象模拟集合，存终点节点下标（此数据集中下标与id相同），O(1)查找

```
var INF = 1e18;
var dist = new Array(n);           // dist[i] = 从起点到节点i的最短时间
var transferArr = new Array(n);    // transferArr[i] = 从起点到节点i的换乘次数
var visited = new Array(n);        // visited[i] = 节点i是否已确定最短路径
var prev = new Array(n);           // prev[i] = 到达节点i的前驱节点id，用于回溯路径
```

● C对照: `c double dist[289]; // JS: var dist = new Array(n) int transferArr[289]; // JS: var transferArr = new Array(n) int visited[289]; // JS: var visited = new Array(n) int prev[289]; // JS: var prev = new Array(n)` ● C 要为每个数组指定元素类型 (double/int) , JS 的 var 统一处理

```
for (var i = 0; i < n; i++) {
    dist[i] = INF;           // 距离初始化为无穷大
    transferArr[i] = 0;      // 换乘次数初始化为0
    visited[i] = false;     // 都未访问
    prev[i] = -1;           // 前驱初始化为-1（无前驱）
}
```

## 4.4 起点入堆

```
var heap = new MinHeap();
```

● 创建最小堆 ● C对照: `MinHeap h; h.size = 0; 或 MinHeap* h = (MinHeap*)malloc(sizeof(MinHeap));` ● JS 的 `new MinHeap()` 自动调用构造函数初始化, C 需要手动初始化或用 `malloc`

```
for (var s = 0; s < startNodes.length; s++) {
    var si = startNodes[s];
    dist[si] = 0;
    transferArr[si] = 0;
```

```
var cost = mode === 1 ? 0 : 0;    // 三种模式起点cost都是0
heap.insert({ nodeId: si, cost: cost, totalTime: 0, transfers: 0 });
}
```

- 把起点对应的所有节点都入堆，距离为0

## 4.5 主循环 —— Dijkstra 的核心

```
var foundNode = -1;
```

- 记录找到的终点节点id，-1表示还没找到

```
while (!heap.isEmpty()) {
  var cur = heap.extractMin();
  if (cur === null) break;
```

- 取出堆中cost最小的节点

```
if (visited[cur.nodeId]) continue;
```

★ **关键：懒惰删除。** 同一个节点可能被多次insert（发现更短路径时），但只有第一次extractMin时是最优的，后续的都跳过

```
visited[cur.nodeId] = true;
```

- 标记当前节点为已访问（已确定最短路径）

```
if (endSet[cur.nodeId]) {
  foundNode = cur.nodeId;
  break;
}
```

★ **关键：到达终点立即退出！** Dijkstra保证第一次到达终点时就是最优的

## 4.6 扩展邻居 —— 松弛操作

```
var edges = adjList[cur.nodeId];
if (!edges) continue;
for (var j = 0; j < edges.length; j++) {
    var edge = edges[j];
    var neighbor = edge.to;
    if (visited[neighbor]) continue;
```

● 取出当前节点的所有邻居边，跳过已访问的邻居

```
var newTime = dist[cur.nodeId] + edge.weight;
```

★ **核心：新距离 = 到当前节点的距离 + 边的权重。** 这就是"松弛"——尝试经过当前节点到邻居是否更短

```
var newTransfers = transferArr[cur.nodeId] + edge.is_transfer;
```

● 新换乘数 = 到当前节点的换乘数 + 这条边是否换乘。is\_transfer=0时不变，=1时换乘数+1

## 4.7 cost 计算 —— 三种模式的核心区别

```
var newCost;
if (mode === 0) {
    newCost = newTime; // 模式0: cost = 纯时间，谁快选谁
} else if (mode === 1) {
    newCost = newTransfers + newTime * 1e-6; // 模式1: 换乘主导，时间极小值打破平局
} else {
    newCost = newTime * 1000 + newTransfers; // 模式2: 时间主导，时间相同时换乘少优先
}
```

● mode=1中，transfers是整数(0,1,2...), totalTime×0.000001极小，所以换乘次数主导 ● mode=2中，totalTime×1000让时间成为主导因素，transfers作为低位

## 4.8 比较并更新

```
if (newCost < (dist[neighbor] === INF ? INF
    : mode === 0 ? dist[neighbor]
    : mode === 1 ? transferArr[neighbor] + dist[neighbor] * 1e-6
    : dist[neighbor] * 1000 + transferArr[neighbor])) {
```

● 比较新cost和旧cost。如果邻居还没被更新过(dist=INF)，新cost一定更小 ● 这里用嵌套三元运算符根据模式计算旧cost，和4.7节的新Cost公式对称

```
dist[neighbor] = newTime;
transferArr[neighbor] = newTransfers;
prev[neighbor] = cur.nodeId;
```

★ **关键：记录前驱**，用于最后回溯路径

```
heap.insert({
    nodeId: neighbor,
    cost: newCost,
    totalTime: newTime,
    transfers: newTransfers,
});
```

★ **懒惰删除策略**：不是修改堆中的旧值，而是插入新值。旧值还在堆里，但extractMin时会因为visited被跳过。避免堆中查找的开销 ● C对照：MinHeap\_insert(&h, (HeapNode){neighbor, newCost, newTime, newTransfers}); ● C用复合字面量 (HeapNode){...} 创建临时结构体，JS用对象字面量 {...} —— 语法几乎一致

```
    }
  }
}
```

## 4.9 路径回溯

```
if (foundNode === -1) {
    return { path: [], error: "No path found" };
}
```

```
}

```

```
var path = [];
var cur = foundNode;
while (cur !== -1) {
    path.unshift(cur);    // unshift = 插入数组头部，最终path从起点到终点
    cur = prev[cur];      // 移动到前驱节点
}
```

- 从终点沿prev往回走，unshift保证顺序是起点→终点

#### 4.10 构造返回结果

```
var pathResult = [];
for (var i = 0; i < path.length; i++) {
    var node = nodes[path[i]];
    pathResult.push({
        station: node.station,
        line: node.line,
        lon: node.lon,
        lat: node.lat,
    });
}
```

- 把节点下标转换为站名、线路、坐标信息

```
var transferStations = [];
var uniqueCount = 0;
var lastStation = "";
for (var i = 0; i < pathResult.length; i++) {
    if (pathResult[i].station !== lastStation) {
        uniqueCount++;
        lastStation = pathResult[i].station;
    }
    if (i > 0 && i < pathResult.length - 1) {
        if (pathResult[i].station === pathResult[i - 1].station &&
            pathResult[i].line !== pathResult[i - 1].line) {
            transferStations.push(pathResult[i].station);
        }
    }
}
```

```

    }
  }
}

```

★ **关键判断**：当前站和前一站站名相同但线路不同 → 换乘！

```

return {
  path: pathResult,
  total_time: dist[foundNode],
  transfers: transferArr[foundNode],
  transfer_stations: transferStations,
  station_count: uniqueCount,
};

```

● 返回完整结果：路径、总时间、换乘次数、换乘站列表、经过站数

## 五、dijkstraWithPenalty —— 带惩罚的Dijkstra（备用方案）

用于生成备用方案。原理：对最优路径上的中间站点施加高惩罚，迫使算法绕开这些站点。

### 5.1 与普通dijkstra的区别

dijkstraWithPenalty 在 dijkstra 基础上增加了：

1. **realDist[] 数组** —— 记录真实时间（不含惩罚）
2. **penalty 参数** —— 每个节点可以有一个惩罚值
3. 计算newCost时，把惩罚值加到时间上

### 5.2 关键代码差异

```
var penalty = (penaltyWeights && penaltyWeights[cur.nodeId]) || 0;
```

● 取出当前节点的惩罚值，没有则为0 ● C对照：double penalty = (penaltyWeights != NULL) ? penaltyWeights[cur.nodeId] : 0; ● JS 的 && 和 || 短路求值 ≈ C 的三元运算符，但更简洁

```
var newRealTime = realDist[cur.nodeId] + edge.weight;
```

- 真实时间 = 不含惩罚的纯行驶时间

```
var newPenalizedTime = dist[cur.nodeId] + edge.weight + penalty;
```

- 惩罚时间 = 上一节点的惩罚时间 + 边权 + 当前节点惩罚
- penalty让经过被惩罚节点的路径cost变大, 算法会尽量绕开

```
newCost = newPenalizedTime; // mode=0时
```

- cost用的是含惩罚的时间, 但返回的total\_time是真实时间

### 5.3 备用方案生成流程 (findAlternativePaths)

1. 先用 `dijkstraWithPenalty(start, end, mode, null)` 求最优路径
2. 提取最优路径上的中间站点
3. 对每个中间站点, 设置 `penalty=10000`, 重新求路径
4. 去重: 如果新路径和已有路径相似度>80%, 视为重复, 丢弃
5. 按时间+换乘排序, 最多返回5个方案

- `penalty=10000` 相当于给该站点加了10000分钟惩罚, 算法几乎不会经过它

## 六、ScheduleConfig —— 时段调度如何影响算法

**ScheduleConfig 模块在 buildAdjList 阶段就介入了, 它修改的是邻接表中的边权。**

### 6.1 工作流程

1. 检测当前时段 (早高峰/晚高峰/平峰)
  - 根据当前时间自动判断, 或手动选择
2. 获取对应系数
  - 高峰: 运行×1.3, 换乘×1.5
  - 平峰: 运行×1.0, 换乘×1.0

## 3. 调整每条边的权重

- 行驶边:  $\text{weight} \times \text{运行系数}$
- 换乘边:  $\text{weight} \times \text{换乘系数}$

## 4. 用调整后的邻接表运行 Dijkstra

- 高峰期换乘5分钟变7.5分钟，算法会更倾向于少换乘

## 6.2 关键代码

```
function adjustWeight(baseWeight, lineName, isTransfer) {
    var factor = getMultiplier(lineName, isTransfer);    // 获取系数（根据时段和是否换乘）
    return Math.round(baseWeight * factor * 100) / 100;    // 新权重 = 原始权重 × 系数，保留两位小数
}
```

● C对照: `c double adjustWeight(double baseWeight, char* lineName, int isTransfer) { double factor = getMultiplier(lineName, isTransfer); return round(baseWeight * factor * 100) / 100; // C 的 round() ≈ JS 的 Math.round() }` ● C 要声明返回值类型 `double` 和参数类型, JS 不需要。  
`Math.round() ≈ C 的 round()`

### 答辩话术:

"时段调度不是在Dijkstra内部处理的，而是在构建邻接表时就把权重调整好了。  
 高峰期换乘系数1.5，所以换乘边从5分钟变成7.5分钟，  
 Dijkstra自然会倾向于少换乘的路径。这是在数据层面的预处理，算法本身不需要改动。"

## 七、答辩速查：被问到某段代码怎么说

### 7.1 "讲一下你的MinHeap实现"

回答模板:

"最小堆用一维数组存储，下标从0开始。

节点i的父节点是 $\text{floor}((i-1)/2)$ ，左子是 $2i+1$ ，右子是 $2i+2$ 。

insert操作: 新元素放末尾，和父节点比较，更小就交换，一直上浮， $O(\log n)$ 。

extractMin操作: 取堆顶，末尾补顶，和较小子节点比较，更大就交换，一直下沉， $O(\log n)$ 。

堆中存的是 $\{\text{nodeId}, \text{cost}\}$ ，cost是Dijkstra的优先级值。"

● 追问"和C语言实现有什么区别"时的补充: "逻辑完全一样。C用结构体+函数传指针: `MinHeap_insert(&h, node)`, JS用构造函数+原型方法: `h.insert(node)`, `this`自动绑定等价于C的`&h`。C的整数除法自动取整, JS需要`Math.floor()`。C手动维护size, JS用`push/pop`自动管理。"

## 7.2 "讲一下你的Dijkstra实现"

回答模板:

"首先从`graph.json`构建邻接表, 每个节点的邻居存在数组里。

然后初始化`dist`数组为无穷大、`visited`为`false`、`prev`为-1。

把起点入堆, 然后循环: 从堆中取`cost`最小的节点,

如果已访问就跳过, 否则标记已访问, 检查是否到达终点。

没到终点就遍历邻居, 计算新的`cost` (根据模式不同公式不同),

如果比原来的小就更新`dist`、`transferArr`、`prev`, 并把邻居入堆。

最后从终点沿`prev`回溯到起点, 得到完整路径。"

● 追问"和C语言实现有什么区别"时的补充: "算法逻辑完全一样。主要语法差异: JS用`var`声明变量不需要类型, C要写`double dist[]`和`int visited[]`; JS用`this`代替C的指针传参; JS的对象字面量 `{nodeId, cost}` 等价于C的结构体实例化 `(HeapNode){nodeId, cost}`。算法思想、数据结构、时间复杂度在两种语言中完全相同。"

## 7.3 "换乘是怎么处理的?"

回答模板:

"换乘站按'站名+线路'拆分为多个节点, 换乘边连接同一站名不同线路的节点,

权重5分钟, `is_transfer=1`。Dijkstra扩展到换乘边时,

换乘次数+1, 时间+5分钟。这样就能同时追踪时间和换乘次数。

路径结果中, 如果相邻两个节点站名相同但线路不同, 就是换乘。"

## 7.4 "三种模式有什么区别?"

回答模板:

"区别在`cost`的计算公式:

`mode=0`时间最短: `cost = totalTime`, 纯比时间。

`mode=1`换乘最少: `cost = transfers + totalTime×0.000001`,

换乘次数主导, 时间极小值打破平局。

`mode=2`综合最优: `cost = totalTime×1000 + transfers`,

时间主导, 时间相同时换乘少的优先。

三种模式用的是同一个Dijkstra, 只是`cost`公式不同。"

## 7.5 "备用方案怎么生成的？"

回答模板：

"用惩罚避让策略。先求最优路径，然后对路径上的中间站点

施加**10000**分钟惩罚，迫使算法绕开这些站点。

新路径和已有路径相似度超过**80%**就丢弃，最终按时间排序展示。"

## 7.6 "为什么图有289个节点而不是250个？"

回答模板：

"因为换乘站按'站名+线路'拆分。比如小寨站属于2号线和3号线，

在图中是两个独立节点，通过换乘边连接。

这样**Dijkstra**才能追踪当前所在线路，正确计算换乘次数和等待时间。"